



Module 28

Introduction to Simulation Acceleration

cādence®

This page does not contain notes.

Module Objectives

In this module, you will be able to

- Define the verification acceleration platforms
 - Multi-Core Simulation
 - Processor-based emulation
 - FPGA-based prototyping
- Distinguish between verification acceleration technologies
 - Compare multi-core simulation, emulation, and prototyping



The objective of this module is to understand verification acceleration platforms. We focus on multi-core simulation, emulation, prototyping, and a general comparison between them. We also cover comparisons based on design run speed.

By going through all these, we will be able to understand why we need emulation and prototyping in addition to simulation. We also understand why emulation is required if prototyping is already there. Based on the scenario, we can also decide whether to use emulation or prototyping.

Cadence products referenced in this presentation are:

- Xcelium MultiCore Simulation
- Palladium emulation
- Protium rapid-prototyping

Problem: Test Suite Latency

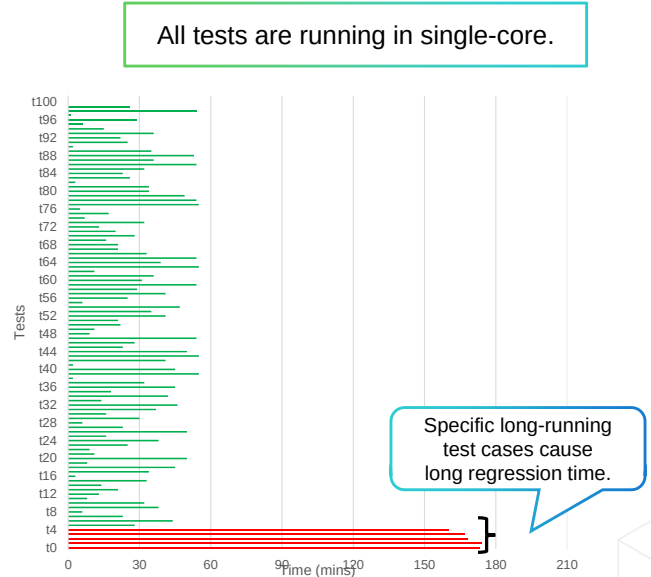


Test suite latency is the delay or time taken by long-running test cases in a test suite or regressions.

Faster simulation gives lower latency, allowing:

- Quicker test-debug-fix-verify cycles
- More complete regression testing
- Longer random tests for better coverage
- Tests that would otherwise be impractical
 - e.g. boot Linux on SoC RTL

Acceleration allows us to run more test cycles on bigger SoCs in less time

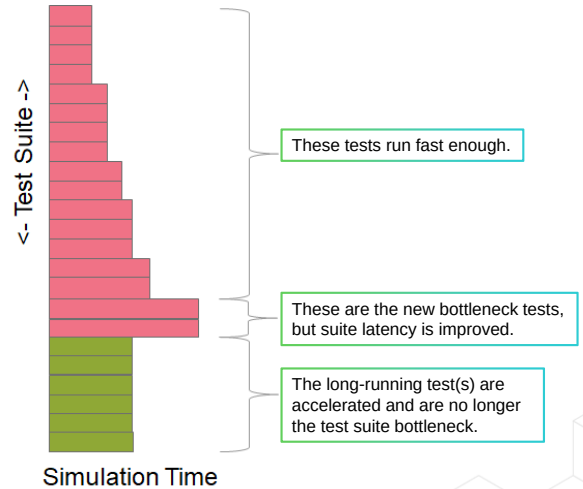
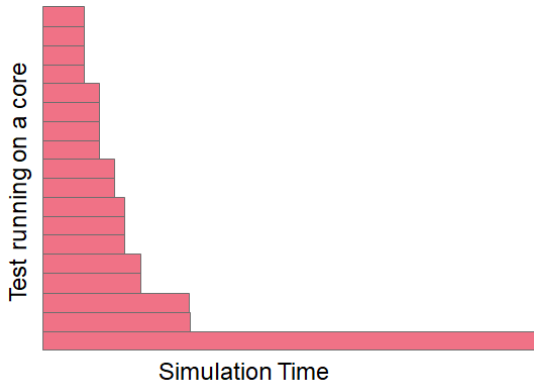


There is rarely enough time to execute all the verification tests we would like. Hard deadlines mean our verification is inevitably compromised. We run fewer tests, longer tests less frequently, or shorter randomization stimuli, leading to potentially lower coverage. More comprehensive tests may be completely impractical due to time considerations.

Imagine what you could do if your tests lasted hours rather than days or days rather than weeks. We can find, debug, fix, and verify bugs much faster. We can apply more complex stimuli, run a full suite of tests during regression testing, run random stimulus tests for longer, or indeed apply tests that would otherwise be totally impractical, such as booting Linux on a predominately RTL model of your DUT.

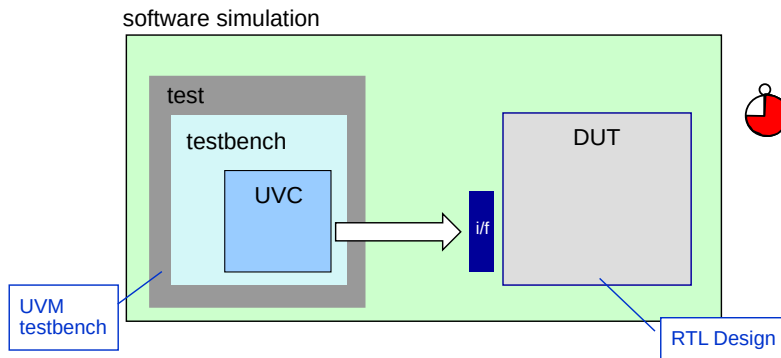
Example: Test Suite Latency Problem

- The long-running test cases in any test suite or regression control test suite latency.
- Verification Teams manage this problem by:
 - Having short critical-functionality tests
 - Running long tests infrequently
- Acceleration solves test suite latency problems by speeding up long-running tests.



This page does not contain notes.

Traditional Simulation

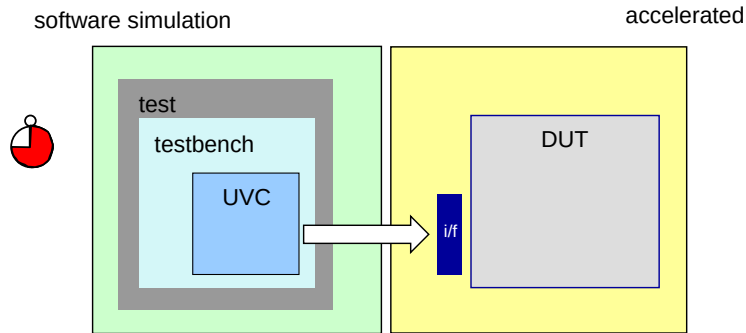


Traditional simulation is done purely in software

- Test environment runs quickly in creating stimulus
- DUT consumes most of the simulation time processing the stimulus

This page does not contain notes.

Simple Acceleration



Simple partition of testbench to simulator and DUT to acceleration platform

Unoptimized testbench limits performance gains

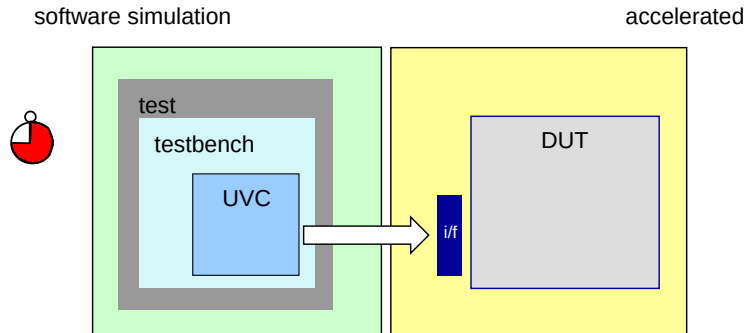
- e.g. Passing stimulus data on every clock cycle limits acceleration performance

Called Signal-Based Acceleration (SBA):

- Can improve simulation performance by an order of magnitude

This page does not contain notes.

Optimized Acceleration



Testbench can be optimized, for example:

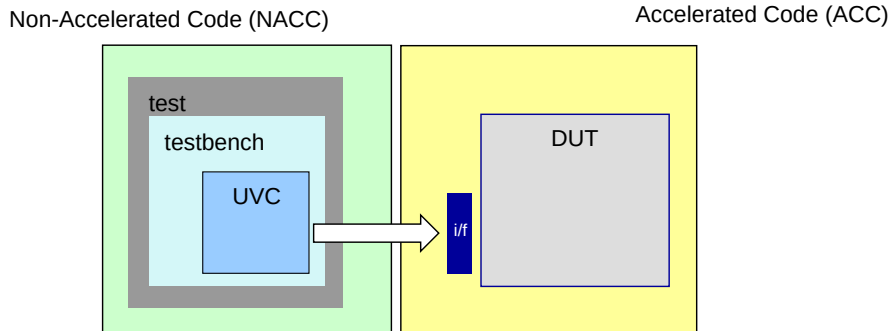
- Pass multiple stimulus data at a time
- Reduce interaction between testbench and DUT

This is Transaction-Based Acceleration (TBA):

- Can improve simulation performance by several orders of magnitude

This page does not contain notes.

Accelerated and Non-Accelerated Code



Non-Accelerated Code (NACC)

- Non-synthesizable code such as testbenches and non-SystemVerilog models

Accelerated Code (ACC)

- Synthesizable RTL Design and gate level code

Generally, only synthesizable code is acceleratable

This page does not contain notes.

Acceleration Technologies Overview

Technology	Basis	Process Outline	Product
Multi-Core Simulation	Software	Design automatically partitioned to run in parallel on multiple server cores	Xcelium Multi-Core
Emulation	Processor	1. Design automatically synthesized to gate level netlist 2. Compiled into runnable instructions 3. Executed on a parallel network of custom processors	Palladium
Rapid-Prototyping	FPGA*	Design automatically partitioned and synthesized into a networked array of FGPA devices	Protium

Emulation and Rapid-Prototyping generally require the design RTL code to be synthesizable

- Palladium synthesizes code into a Gate Level Netlist to compile into processor instructions
- Protium synthesizes code into an FGPA

Multi-core simulation has similar restrictions

*Field Programmable Gate Array



This page does not contain notes.

Comparison Among Simulation, Emulation and Prototyping

Property	Single-Core Simulation	Emulation	Prototyping
Time to bring-up	Days	1-2 weeks	1-2 weeks
Design Style Support	All	DUT should be synthesizable	DUT should be synthesizable
Run time speed	1-10 KHz	0.5-2 MHz	2-10 MHz
Visibility (for debug)	All signals	All signals	All signals
Real Targets	No	Yes	Yes
Linux Boot Time	Week	Under a day	Few hours



10 © Cadence Design Systems, Inc. All rights reserved.

cadence

This slide compares single-core simulation, emulation, and prototyping based on various properties and shows the diagrams of computing hardware used, respectively.

For the same design, the time to bring up in simulation is typically a few days but a few weeks for emulation and prototyping.

Simulation supports all kinds of design styles, but the DUT should be synthesizable for emulation and prototyping.

The run-time speed in simulation is in the range of 1 to 10 KHz. In emulation, it's in the range of 0.5 to 2 MHz, and in prototyping, in the range of 2 to 10 MHz.

Emulation and prototyping are approximately 1000 times faster than simulation.

For debugging, the visibility of all signals is possible in simulation, emulation, and prototyping, but in prototyping, the resource utilization is more than emulation.

The real targets can be used in emulation and prototyping but not in the simulation platform.

Also, for SOC design having a Linux operating system, the boot time is in weeks for simulation, but it's only a few hours in emulation and prototyping platforms.

The pictures at the bottom of the table represent that any general-purpose x86 machine can be used for simulations. Cadence emulation runs on Palladium hardware, and Cadence Protium can be used for prototyping. Palladium and Protium systems are designed and built by Cadence.

In summary, the simulation is very flexible, and visibility is very high, but it doesn't scale after a design crosses some size. You can't find a machine large enough to compile your designs. After a particular size, we must use emulation or prototyping. In the Cadence Palladium emulator, we have thousands of parallel custom processors that run millions of instructions per clock, which makes it very fast, and we can use real software to run in chips under development.

Typical Use of Simulation, Emulation and Prototyping

1. Simulation

- Suitable for small design blocks
- Applicable when a design size is approximately less than 50 million gates
- Takes hours to complete
- Used when we need to run 5-6 iterations per day

2. Emulation

- Ideal for full chip or major subsystems integration
- Necessary for debugging RTL and generating waveforms for a large number of cycles
- Allows testing with real targets (e.g., PCIe card, Display) at reduced speed

3. Prototyping

- Used to bring up software on a chip under development
- Enables testing with real targets at full speed
- Facilitates software tests that take days

11 © Cadence Design Systems, Inc. All rights reserved.



This slide shows the typical use of simulation, emulation, and prototyping platforms.

The simulation is used when we need to test the small design blocks, or the design size is less than 50 million gates.

This is also recommended when a complete test case is executed in some hours, and we need five to six iterations daily.

The emulation is used mainly for the full chip or major subsystem integration.

This is recommended when we need to debug RTL and need waveforms for a large number of cycles.

This is also used when our purpose is to use real targets like PCIe cards or Displays, and working at a reduced speed is acceptable.

Prototyping is used when we need to bring up software on a chip under development or do software testing.

This is also recommended when we need to test with real targets at full speed and software tests, which take many days.

Prototyping trades increased speed for lower signal visibility and debugging compared to emulation.

What Is Design Run Speed?



The speed at which the RTL Design runs on a selected platform is called the Design Run Speed.

- Design Run Speed determines how fast the designer can see the result of a test case execution
 - Influences how quickly a designer can detect/debug an issue.
- For example, here are indicative numbers for the same design with similar test cases on different platforms:

Platform	Typical Design Run Speed	Execution time (for similar test case)
Single-Core Simulation	10 KHz	60,00,000 seconds (1666 hours)
Emulation	1 MHz	60,000 seconds (16.67 hours)
Prototyping	4 MHz	14,760 seconds (4.1 hours)



This slide introduces design run speed.

The speed at which the RTL design runs in the selected platform is called the design run speed on that platform.

The design run speed will decide how fast the designer can see the result of a test case execution. It is an important criterion and influences how quickly a designer can detect/debug an issue.

For example, if the typical speed for the design in simulation is 10 KHz. The same design can run in emulation with a speed of 1 MHz, and in prototyping, it can run with 4 MHz. These numbers are indicative only.

Similarly, the execution time for the same test case will be 1666 hours in simulation but around 17 hours in emulation and 4 hours in prototyping.

What Is Xcelium Multi-Core?



Xcelium MC is a simulation engine leveraging multi-core servers to parallelize the simulation.

It offloads the most time-consuming calculations from the single-core engine to an engine that runs over multi-core CPUs.

- Multi-Core Simulator reduces simulation time by:
 - Breaking the design into independent parts
 - Simulating those parts on parallel cores of a server
- It automatically identifies areas of code that can be accelerated
- It enables a tradeoff of more resources (cores) to accelerate long tests
- Not all portions of code can be accelerated by multi-core



Selective long-running tests run on multi-cores, resulting in faster regression turnaround time.

This page does not contain notes.

Features of Xcelium Multi-Core

The Xcelium Multi-Core comes with the following features:

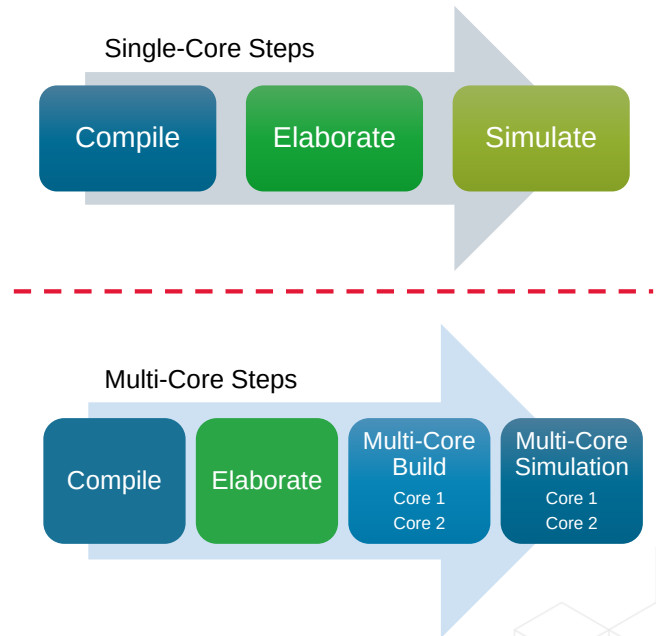
- Supports the entire SystemVerilog language.
 - Reduces SystemVerilog simulation time.
 - Breaks the design dependencies into independent parts and then simulates on parallel cores of a server.
 - Provides significant acceleration of simulation across active and large designs.
- Supports large, highly active gate-level designs.
- Supports and accelerates functional Gate-Level, zero delay, and Standard Delay Format (SDF) simulations.
- Runs Automatic Test Pattern Generation (ATPG) vector simulation.
- Accelerates functional gate-level designs where the DUT (rather than the testbench) dominates the simulation profile.



This page does not contain notes.

Xcelium Single Core Versus Multi-Core Simulation

- The Xcelium MC uses compile and elaborate steps similar to Xcelium single-core flow.
- After elaboration, the MC build process starts, and the final output is a snapshot.
- The final snapshot is used for multi-core simulation.



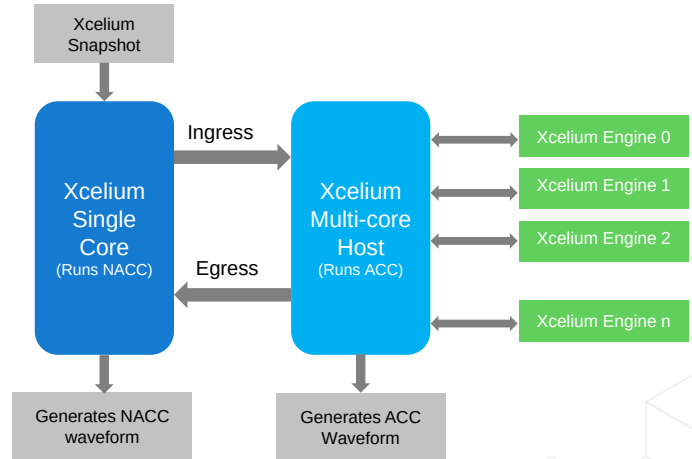
This page does not contain notes.

How Xcelium Multi-Core Simulation Works?



The Xcelium Multi-Core engine is a CPU-based accelerator for the Xcelium Simulator integrated with the existing single-core simulator.

- A single-core host runs the non-accelerated portions of the design.
- A multi-core host occupies a single core, and engines occupy the rest of the cores.
- The multi-core engines run the accelerated portions of the design.
- The number of cores available for Xcelium MC is configurable



This page does not contain notes.

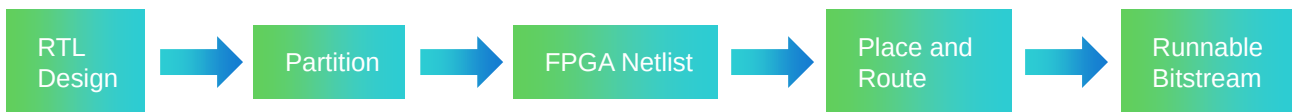
What Is FPGA-Based Prototyping?



FPGA-based prototyping involves hardware that implements RTL design into high-capacity FPGAs.

This is used to speed up the test case execution for RTL testing.

- Protium is an FPGA-based prototyping system built by Cadence.
- Protium uses some of the largest FPGAs available
 - For example, Protium X2 uses Xilinx Vu19p FPGA.
- The sequence of operations from RTL design to its implementation in hardware is:



This slide introduces FPGA-based prototyping. FPGA-based prototyping involves hardware that implements RTL design into standard FPGAs. This is used to speed up the test case execution for RTL testing.

Protium is an FPGA-based prototyping system built by Cadence. It contains the largest FPGAs in the world, such as the Xilinx Vu19p FPGA.

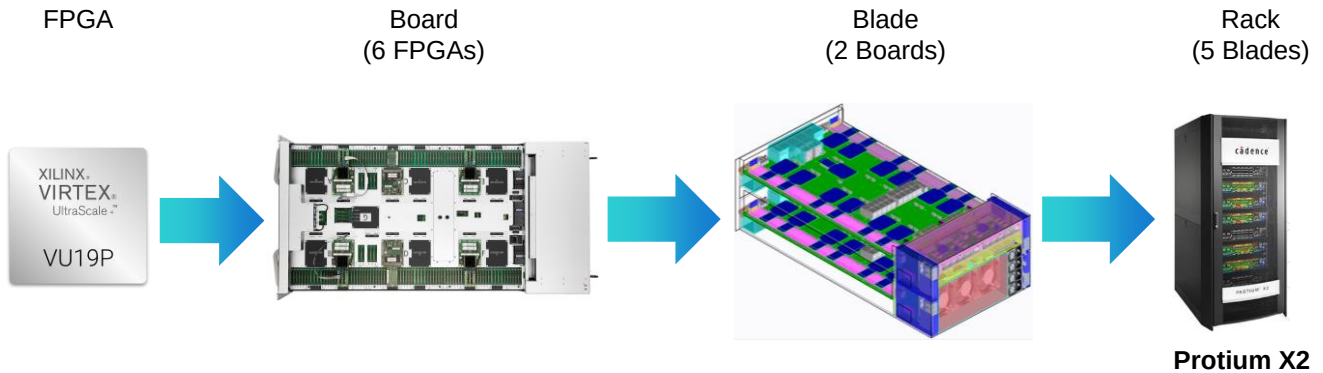
The first step for the hardware implementation after RTL design is a partition of RTL into multiple FPGAs.

The second step is the generation of an FPGA netlist for each FPGA separately.

The third step is the Place and route for each FPGA separately.

The fourth step is to generate a runnable bit stream from the placed and routed netlist for each FPGA involved, which will finally run on FPGAs in the Protium hardware.

Protium Structure



1 rack of Protium X2 can emulate up to 1.2 billion gates, and 4 Protium X2 racks can be wired together.

18 © Cadence Design Systems, Inc. All rights reserved.

cadence

This slide represents the structure of the Protium emulator, starting from the FPGA, which is a basic building block of Protium.

The FPGA contains logic cells that implement the RTL functionality. Six FPGAs are connected to make a board.

Two boards are connected to make a blade, and five blades are connected to make a rack. The Protium X2 shown in the diagram is made up of one rack.

For example, one rack of X2 can emulate up to 1.2 billion gates, and 4 Protium X2 racks can be wired together to work as one big Protium X2 system.

Prototyping Features



Prototyping is typically used when:

- The chip is typically larger than a hundred million gates.
- The chip is used with large software stacks which have long compute times, such as:
 - Processors working with operating systems
 - Graphics and gaming
 - Machine learning
 - Network traffic handlers
- The system chips are built for time-critical applications.
 - For example, testing camera chip data with real-time software processing.
- We need to connect real targets (e.g., displays/network cards) that can run up to full speed.



The following two slides summarize prototyping usage and limitations.

Prototyping is used when a chip is large, typically larger than a hundred million gates.

This is also recommended when a chip is used with large software stacks with long compute times. These are used in domains such as processors working with operating systems, graphics/gaming, machine learning, and network traffic handlers.

This is also used when the system chips are built for time-critical applications, for example, testing camera chip data with real-time software processing.

It is highly recommended when we need to connect the real targets, for example, display or network cards, even though at reduced speeds.

Prototyping Limitations

Prototyping limitations include:

- Time for compilation and place and route
 - Building a model for a billion-gate design takes:
 - Compile time around 12-18 hours
 - Place and route time around 12-18 hours
- Not effective in testbench-based verification where testbench generates the stimulus.
- Logical mistakes require you to restart from scratch
- Visibility of design signals is lower
- If the design model is not stable, it is better to use an emulator
 - For better signal visibility and faster turnaround time
- Expensive for very small designs
 - For designs under a few million gates the cost is not justified

20

© Cadence Design Systems, Inc. All rights reserved.



This slide covers the prototyping limitations.

Even though your regression in prototyping takes less time, it takes more time to compile your design, and visibility is also lower.

For instance, compile time for a big design can be from 12 to 18 hours. Similarly, the place and route can also take 12 to 18 hours.

This is not effective in testbench-based verification, where the testbench generates the stimulus.

If there is any logical mistake, you need to restart from scratch, which has a big turnaround time.

The visibility of internal signals is also less compared to emulation and simulation. This is only suitable if the hardware model is stable. In this case, emulation is recommended.

Prototyping is very expensive for small designs, which are typically lesser than a few million gates. For smaller designs, the cost is not justified.

What Is Processor-Based Emulation?



Processor-based emulation involves hardware that implements the RTL design into a network of parallel processors.

This is used to speed up the test case execution for RTL testing.

- Palladium is a processor-based emulator designed and manufactured by Cadence.
 - Palladium uses a custom processor network chip built by Cadence
 - This custom chip is one of the largest and most complex in the world.
- The structure is massive and is based on multiple processor units
- The sequence of operations from RTL design to its implementation in hardware is:



This slide introduces processor-based emulation. Processor-based emulation involves hardware that implements RTL Design into the bulk of parallel processors. This is used to speed up the test case execution for RTL testing.

Palladium is a processor-based emulator designed and manufactured by Cadence. It uses a custom processor built by Cadence. This custom processor chip is one of the world's largest and most complex chips in production.

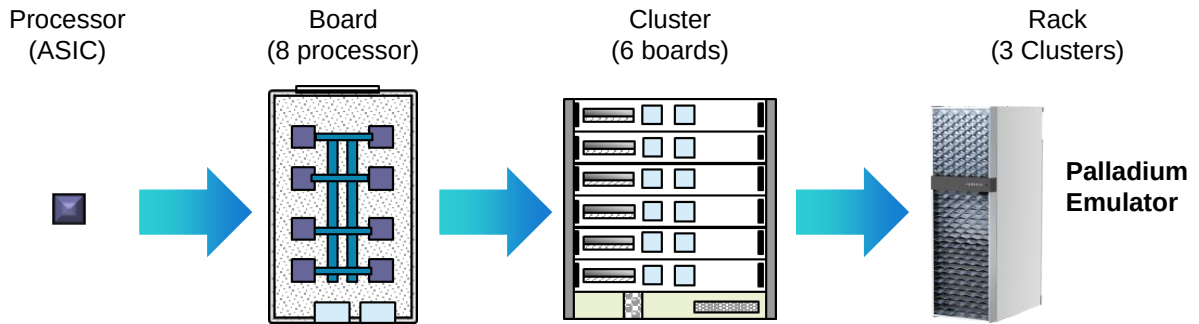
The structure is massive and is based on multiple units of processor structure, which will be shown in the next slide.

The first step for the hardware implementation after RTL design is a synthesis, which converts RTL into a gate-level netlist.

The second step is the compilation of synthesized RTL.

The third step is the conversion of compiled RTL into runnable instructions, which will finally run on parallel processors present in emulator hardware.

Palladium Structure



One rack can emulate 1 billion gates, and up to 8 racks of Palladium can be wired together.

This slide represents the structure of the Palladium emulator, starting from the processor, which is a basic building block of the emulator.

The processor is an ASIC chip that contains a bulk of small processor units which can run many instructions in parallel. Eight such processors are connected to make a board.

Six boards make a cluster, and three clusters make a rack. The Palladium emulator in the diagram contains one rack.

For example, one rack can emulate 1 billion gates, and 8 racks of Palladium emulators can be wired together to work as one big Palladium emulator.

Palladium Features



- Large Capacity
- Flexible, Multi-user System
 - Supports independent user operation with dynamic capacity allocation.
- Fast Compile time
 - Takes an hour to compile a 30 million gate design, giving a quick turnaround time for each design change.
- Debug Capabilities
 - Allows full visibility of any signal and extensive trace paths spanning millions of cycles.
 - Uses any signal at runtime for triggering without recompilation.
 - Supports an easy-to-use State Definition Language (SDL) for trigger management.
- Advanced Capabilities
 - Accelerates mixed-signal design simulations by supporting Real Number Modeling (RNM).
 - Supports 4-State Emulation to accelerate simulations requiring X-propagation with low-power validation.
 - Provides Dynamic Power Analysis (DPA) for in-depth power consumption analysis.
- Other Capabilities
 - Facilitates low power verification with support for standard power formats.
 - Supports Assertion Based Verification (ABV).
 - Supports Universal Verification Methodology (UVM).
 - Facilitates Metric Driven Verification (MDV).

23 © Cadence Design Systems, Inc. All rights reserved.



The slide shows the various Palladium features.

It has a very large capacity, so it is suitable for big designs.

It is a flexible and multi-user system.

The Palladium system considers each user as independent, and its capacity is dynamically allocated to executing jobs.

It has a fast compile time, and typically, one hour is needed for a 30 million gate design. So, there is a quick turnaround time for each design change.

The Palladium system supports many debug capabilities.

It supports full visibility of design signals with long trace paths of millions of cycles.

Any signals can be used to trigger at runtime without recompilation.

State definition language is also supported to control the triggers.

The Palladium system also has many advanced capabilities. Some are mentioned here.

It supports real number modeling to speed up simulation on mixed-signal designs.

It supports four-state emulation to accelerate simulations requiring x-propagation for low-power validation.

It also has the capability of dynamic power analysis.

The Palladium system also has some more capabilities.

Common power and unified power formats are supported for low-power verification.

It also supports assertion-based verification, UVM, and metric-driven verification.

Palladium Limitations

Palladium limitations include:

- Recompiling the whole design is necessary if RTL changes are made, as incremental compile is not supported
 - Unlike in simulation.
- Supports software validation, though not as rapidly as prototyping.
- Offering limited support for SystemVerilog behavioral constructs.



The Palladium has some limitations along with many features.

If RTL changes are done, then recompilation is required for the whole design. Incremental compile is not supported like in simulation.

Software validation is supported but not as fast as the prototyping system.

It also has limitations in the support of behavioral constructs of the SystemVerilog language.

What Is ICE Mode?



ICE stands for In-Circuit Emulation.

ICE is used when the entire testbench is synthesizable.

ICE is used when the design interacts with real targets, and no testbench is running on the host.

In ICE mode:

- This mode is used mainly when both DUT and testbench are synthesizable.
- This mode is needed to verify a design that works with real targets at full speed.
- Entire design runs in the Palladium or Protium systems.
- ICE flow is also referred to as Legacy flow.

This slide explains ICE mode. ICE stands for “in-circuit emulation”.

It is also used when the entire testbench is synthesizable.

It is also used when the design interacts with real targets, and no testbench is running on the host.

The ICE mode is used mainly when the entire design is synthesizable, including DUT and testbench.

This mode is needed when we have to verify the design that works with real targets at full speed.

The whole design runs on processors in the Palladium system or FPGAs in Protium. This is also called “Legacy flow.”

What Is IXCOM Mode?



IXCOM stands for **Incisive® Xcelerator compiler**.

IXCOM is used when the testbench is not fully synthesizable.

IXCOM also supports real targets.

In **IXCOM** mode:

- Main goal is to accelerate testbench-driven verification where testbench primarily runs on a host machine.
- IXCOM helps to synthesize some portions of a SystemVerilog testbench, including:
 - For example, SystemVerilog Assertions
- Synthesizable part (DUT and partial testbench) of the design run on the Palladium system.
- Non-synthesizable part of testbench runs on the host machine or workstation.

This slide explains I X COM mode. I X COM stands for “Incisive Xcelerator compiler.”

I X COM is used when the testbench is not fully synthesizable. It also supports real targets.

In I X COM mode, the main goal is to accelerate testbench-driven verification, where testbench primarily runs on a host machine.

I X COM helps to synthesize a portion of the testbench, which includes system Verilog DPI calls, system tasks like dollar display family of tasks, and system Verilog assertions, etc.

In I X COM mode, the synthesizable part of the design runs on the Palladium system, which includes DUT and a partial testbench.

The non-synthesizable part of the testbench runs on the host machine or workstation.

In I X COM, control is through the testbench and commands running on the simulator.

Comparing ICE and IXCOM

Property	ICE	IXCOM
Compile time	Fastest compile.	Faster than simulation but slower than ICE.
Compile resources used	Smallest compile footprint (CPU, disk, memory).	Testbench is also compiled along with the design; hence it takes more resources.
Runtime resources used	Smallest run-time footprint on a workstation. Control interface only, no simulator used.	Testbench runs on the simulator on the host machine. Host machine requirements depend on testbench size.
Non-synthesizable testbench	Not supported.	Supported.
Runtime controls/commands	Start/stop of execution, force, release, deposit, memory upload/download, etc.	SystemVerilog C-code interface, assertions, coverage, etc., in addition to ICE commands.
Others	Supports DPA (Dynamic Power Analysis) and Low Power Emulation.	Supports DPA, RNM (real number modeling, 4-state and Low Power Emulation).

27 © Cadence Design Systems, Inc. All rights reserved.



This slide compares ICE and I X COM modes for different properties.

The compile is the fastest in ICE. I X COM compile is faster than simulation but slower than ICE.

The compile compute resources required for compilation are minimal in ICE, but I X COM requires more compute resources because testbench is also compiled along with design.

The runtime compute resource required for ICE is minimal as it doesn't need a simulator.

The runtime compute resource required for I X COM is more than ICE as it involves a testbench to run on a host machine simulator.

The host machine resource requirements depend on the testbench size.

The non-synthesizable testbench is supported in I X COM mode but not in ICE mode.

The runtime control/commands supported by ICE are start and stop of execution, force, release, deposit, memory upload/download, etc.

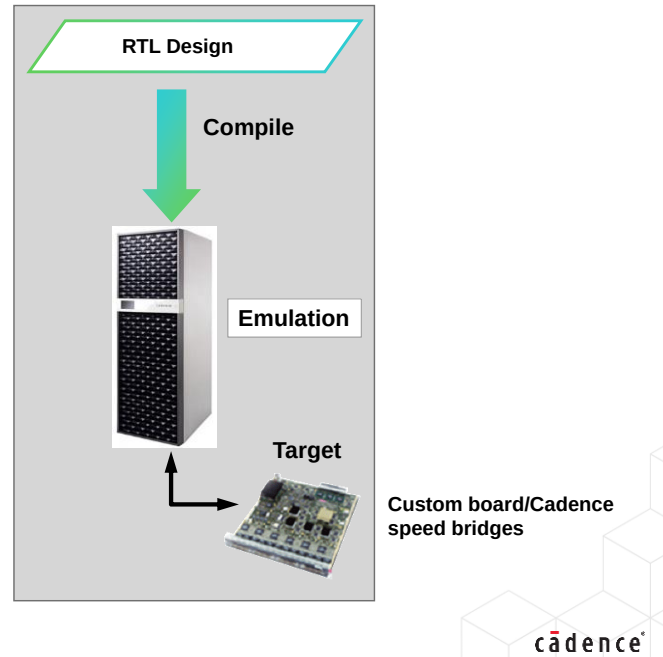
The I X COM supports system Verilog DPI, assertions, coverage, etc., and all the runtime control/commands supported by ICE.

Some other features of ICE are dynamic power analysis and low power emulation. I X COM supports both of these, along with read number modeling and four state.

Emulation (Palladium and Protium) Usage in ICE FLOW

Typical Use Cases

- System-level debugging – with embedded software.
- Stress test of the final design with real-world targets.
- Software development and testing prior to silicon tapeout.
- Provides pre- (and post-) silicon debug platforms.
- Customers can create custom boards to interact with their design running on the emulation system.
- Cadence also provides speed bridges for standard interface protocols.



28 © Cadence Design Systems, Inc. All rights reserved.

This slide shows the practical usage of emulation in ICE flow and typical used cases.

The diagram shows that the whole user design is loaded into Palladium/Protium hardware after compilation. We can also connect real targets to the System, which can be a custom board or Cadence speed bridge.

The typical use cases of the emulation system are as mentioned below.

- It is used for system-level debugging with embedded software.
- The other used case is a stress test of the final design with real-world targets.
- It is also used for software development and testing before silicon tapeout.
- In addition, it also serves the purpose of pre- and post-silicon debug platforms.

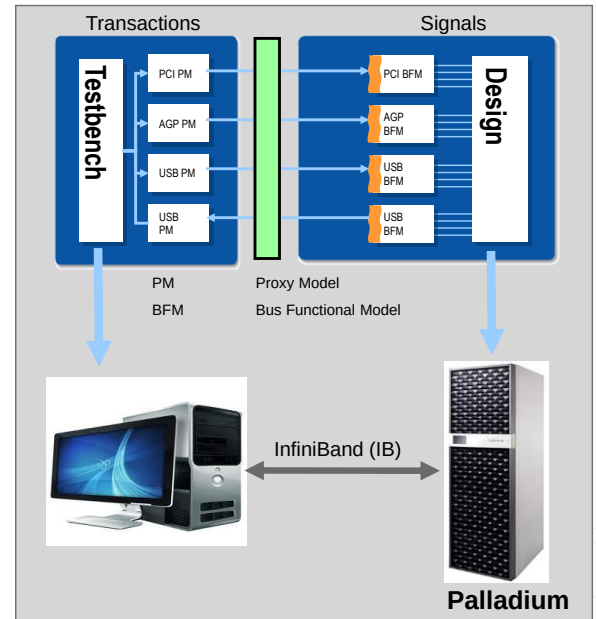
Also, customers can create custom boards to interact with their design while running on the emulation systems.

Cadence also provides speed bridges that can interface with Protium and Palladium systems.

IXCOM (for Transaction-Based Acceleration)

Features

- Transaction-based acceleration (TBA) moves part of the testbench into the Palladium hardware.
- Interface between simulator and Palladium System consists of function calls rather than individual signal transitions.
- Reduces frequency of synchronization between simulator and Palladium System.
- May reduce the amount of data that needs to be transferred.
- Infiniband is a network interface for high-speed communication between the host and the Palladium system. It is necessary for IXCOM designs.



This slide shows the practical usage of Palladium in I X COM flow and its features.

As shown in the diagram, I X COM is used for transaction-based acceleration. The testbench runs in the host machine, and the design is implemented on the Palladium System.

Testbench and design interact through InfiniBand, and the physical connection between them and logical blocks interacted through this connection are also shown here.

The features of this application are as mentioned below.

TBA moves part of the testbench into Palladium hardware, and an interface between the simulator and the Palladium System consists of function calls rather than individual signal transitions.

Transaction-based communication reduces the synchronization frequency between the simulation and the Palladium System.

The TBA approach may reduce the amount of data that needs to be transferred.

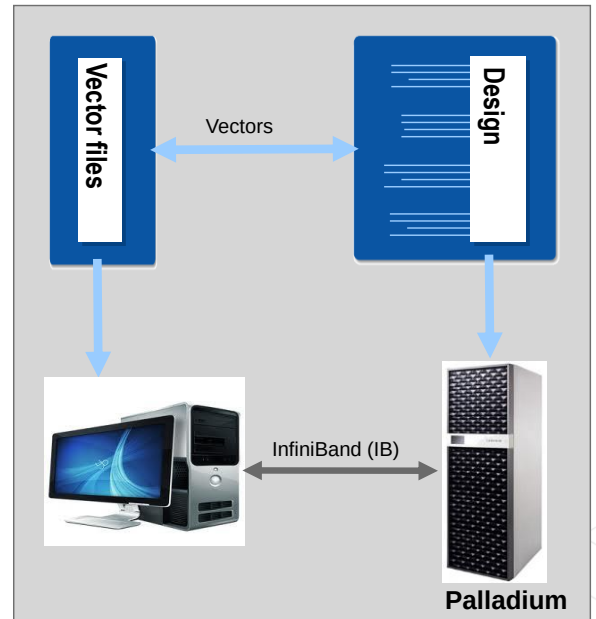
The Infiniband is necessary for the I X COM interface. This is a network interface for high-speed communication between the host and the Palladium system.

If this interface is used in ICE design, it will also improve the performance of runtime commands.

Vector Based Acceleration (Customized Palladium Application)

Features

- Vectors are captured or generated using testability tools, which are used as a stimulus for design.
- Industry standard vector formats like STIL, WGL, etc., are supported by the Palladium system.
- Stimulus is given as input to Palladium.
- Design output is captured and compared with the expected output from the vector files.
- Main use is to validate test vectors for silicon testing.



30

© Cadence Design Systems, Inc. All rights reserved.

cadence

This slide shows the customized application of Palladium in validating vectors.

The diagram shows that Vector files are transferred to the design implemented on the Palladium System. Vector files are transferred to the Palladium machine through InfiniBand.

The vectors are captured or generated using testability tools and used as stimulus.

The industry standard vector format like STIL, WGL, etc. are supported by Palladium system. The stimulus is given as input to Palladium.

The output is captured from design and are compared with expected results from the vector file. Main use of the application shown here is to validate test vectors for silicon testing.

Summary

In this course, you learned to

- Identify various verification platforms
- Compare emulation and prototyping
- Interpret the speed at which your design runs on Palladium®
- Explain the Protium™ and Palladium hardware structure
- Compare ICE and IXCOM compile flow



This page does not contain notes.